

AI 2002

Artificial Intelligence

Lecture 9

Mahzaib Younas

Lecturer, Department of Computer Science

FAST NUCES CFD

Types of Hill Climb

- There are three types of hill climb search
 1. Simple Hill Climb Search
 2. Steepest Ascent Hill Climb Search
 3. Stochastic Hill Climb Search

Simple Hill Climb Search

- This version evaluates neighboring solutions and selects the first one that improves the current state.
- Example:
 - optimizing delivery routes might pick the first alternate route that shortens delivery time, even if it's not optimal.

Steepest Ascent Hill Climb Search

- It is designed to address one of the limitations of the simple hill climbing approach
- This variant assesses all neighboring solutions, choosing the one with the most significant improvement.
- In allocating resources, for instance, it evaluates all possible distributions to identify the most efficient one.

Stochastic Hill Climb Search

- **Stochastic Hill Climbing** doesn't look at all its neighboring nodes to check if it is better than the current node instead, it randomly selects **one neighboring node**, and based on the **pre-defined criteria** it decides whether to go to the **neighboring node or select an alternate node**.

Adversarial Search

- **Competitive** environments, in which the **agents' goals are in conflict**, giving rise to **adversarial search** problems—often known as **games**
 - **Why do AI researchers study game playing?**
- It's a **good reasoning problem**, formal and nontrivial.
- Offer an opportunity to study problems involving **{hostile, adversarial, competing} agents**.
- **Direct comparison** with humans and other computer programs is easy.

Adversarial Search

Mainly games of strategy with the following characteristics:

- Sequence of **moves** to play
- Rules that specify **possible moves**
- Rules that specify a **payment** for each move
- Objective is to **maximize** your payment

Games

- One player wins and the other loses
 - A zero-sum game is defined as one where the total payoff to all players is the same for every instance of the game. Chess is zero-sum because every game has payoff of either $0 + 1$, $1 + 0$ or $\frac{1}{2}$, $\frac{1}{2}$.
- Perfect Information:
 - Players knows the results of the all previous moves
 - There is one best way to win the game for all players
- Imperfect Information
 - Players do not know all of the previous moves

Games

- **Initial State (s_0):**
 - The initial state, which specifies how the game is set up at the start.
- **Players:**
 - defines which player has the move in a state.
- **Actions:**
 - The set of legal moves.
- **Result (s, a):**
 - The transition model, which defines the result of a move. The state after action a is the state s .
- **Terminal Test:**
 - A terminal test, which is true when the game is over and false otherwise.

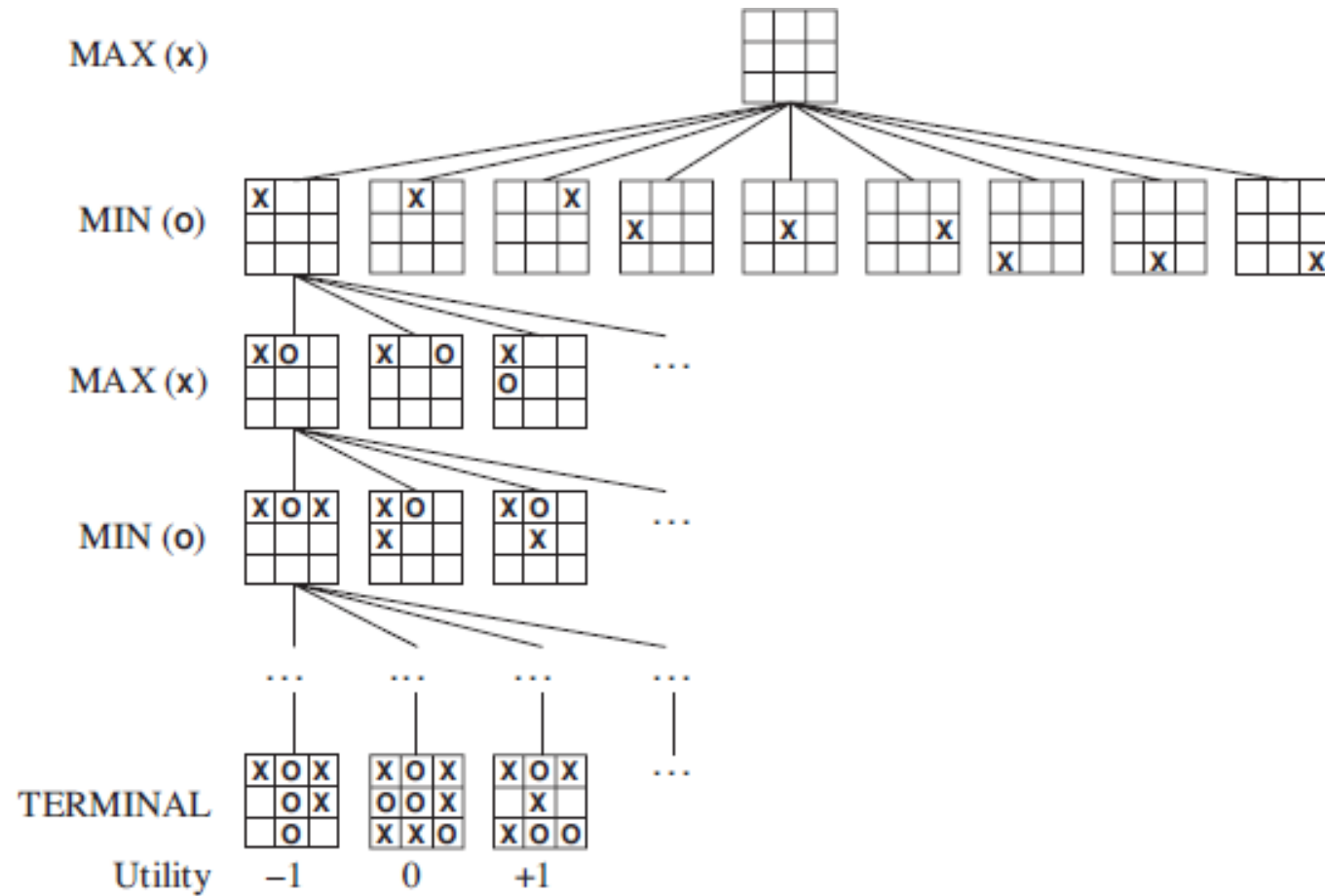
Games

- **Terminal State:**
 - States where the game has ended are called terminal states.
- **Utility:**
 - A utility function (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p .
- In chess, the outcome is a win, loss, or draw, with values +1, 0, or $\frac{1}{2}$.
- Some games have a wider variety of possible outcomes; the payoffs in **backgammon range from 0 to +192.**

Game Tree

- The **initial state**, **actions**, and **results** define the game tree for the game.
- A **game tree** where the **nodes are the game states** and **the edges are moves**.
- The game tree is best thought of as a theoretical construct that we cannot realize in the physical world.
 - For **tic-tac-toe** the game tree is relatively small—fewer than $9! = 362,880$ terminal nodes.
 - For **chess** there are over 10^{40} nodes,

Game Tree : Tic Tac Toe



Mini-Max

- Minimax is a method used **to evaluate game trees.**
- Given a game tree, the optimal strategy can be determined from the **minimax value** of each node.
- *A static evaluator is applied to leaf nodes, and **the values are passed back up the tree to determine the best score** the computer can obtain against a rational opponent.*

Mini-Max

MAX

- Wants to **maximize** the result of the utility function
- **Winning strategy** if, on MIN's turn, a win is obtainable for MAX for all moves that MIN can make

MIN

- Wants to **minimize** the result of the utility function
- **Winning strategy** if, on MAX's turn, a win is obtainable for MIN for all moves that MAX can make

Mini-Max

MAX

Starting value of Max is $-\infty$

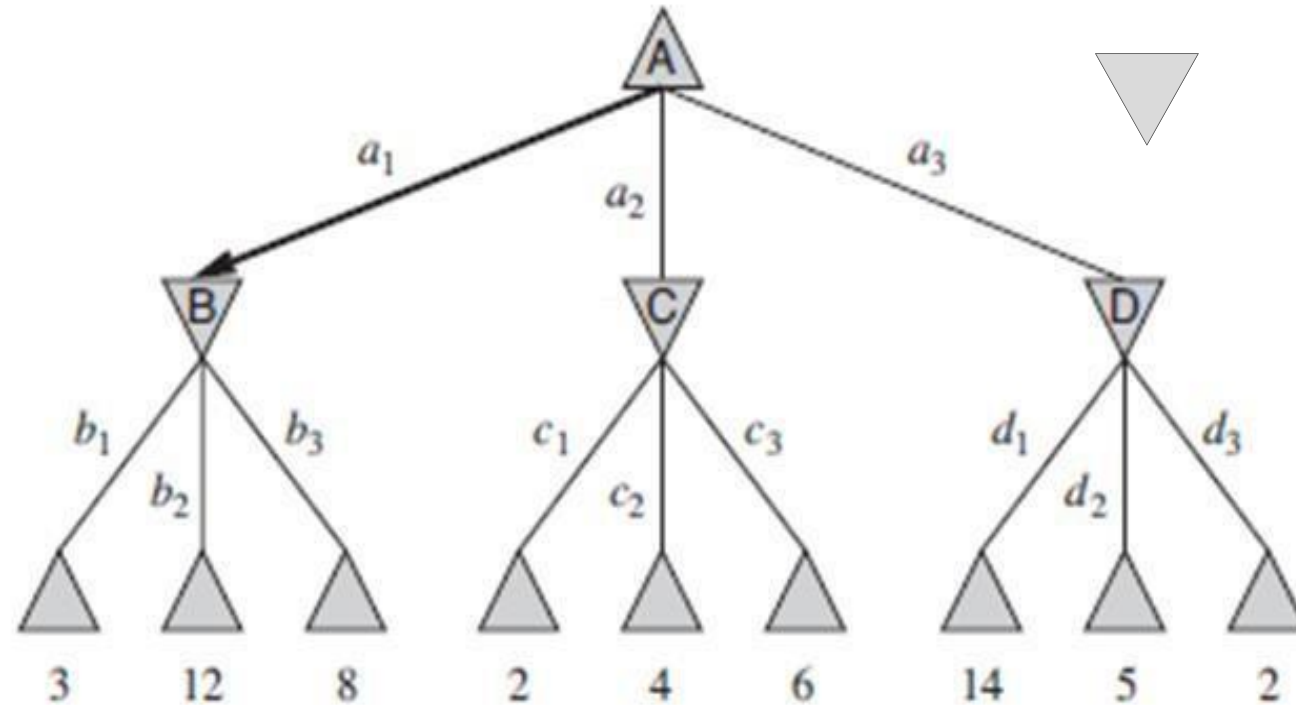
MIN

Starting value of Min is $+\infty$

Mini-Max

MAX

MIN



MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Mini-Max Algorithm

```
function MINIMAX-DECISION(state) returns an action  
  return  $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(state, a))$ 
```

```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

Properties of Min-Max

Complete

- Yes (if tree is finite)

Optimal?

- Yes

Time complexity?

- $O(b^m)$, m is the maximum depth of the tree and b is the legal moves.

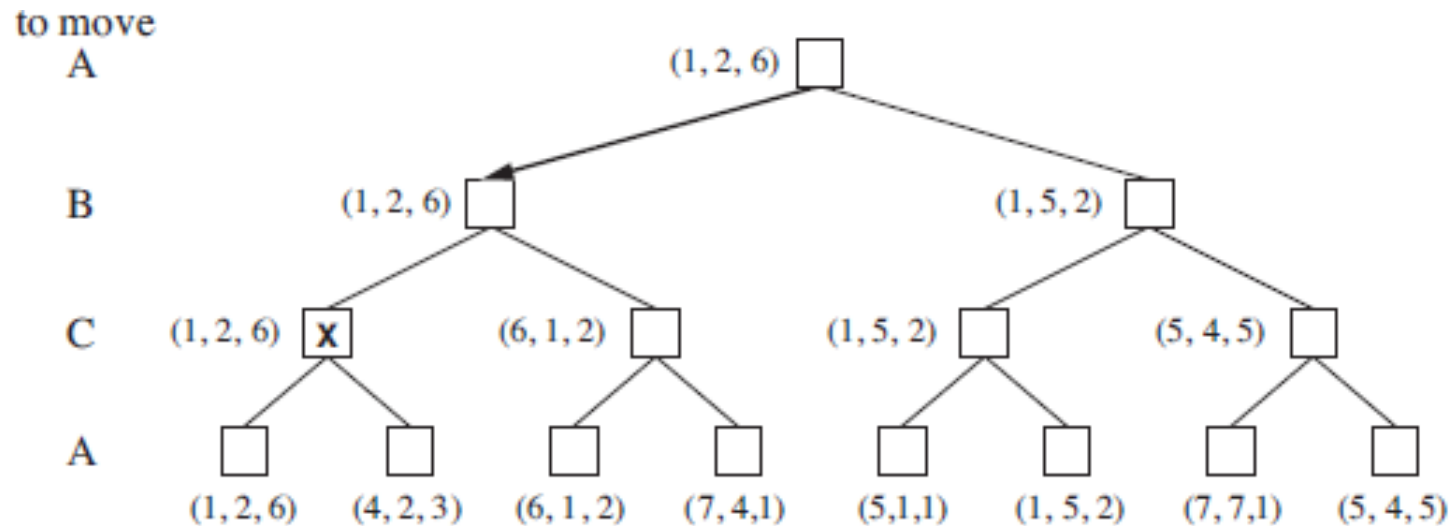
Space complexity?

$O(bm)$ (depth-first search, generate all actions at once)

$O(m)$ (backtracking search, generate actions one at a time)

Multi Player Games

- Each node must hold **a vector of values**
 - **For example**, for three players A, B, C (V_A, V_B, V_C)
- The backed up vector at **node n** *will always be the one*



Pruning

- We can use a **branch-and-bound technique** to reduce the number of states that must be examined to determine the value of a tree.
- **Branch-and-bound Technique:**
- We keep track of a **lower bound on the value of a maximizing node**, and don't bother evaluating any trees that cannot improve this bound.
- Keep track of an **upper bound on the value of a minimizing node**. Don't bother with any sub-trees that cannot improve this bound.

Mini-Max Alpha Beta Cutoffs

- **Alpha Cutoffs:**

- Alpha is the *lower bound on maximizing nodes.*

- **Beta Cutoffs:**

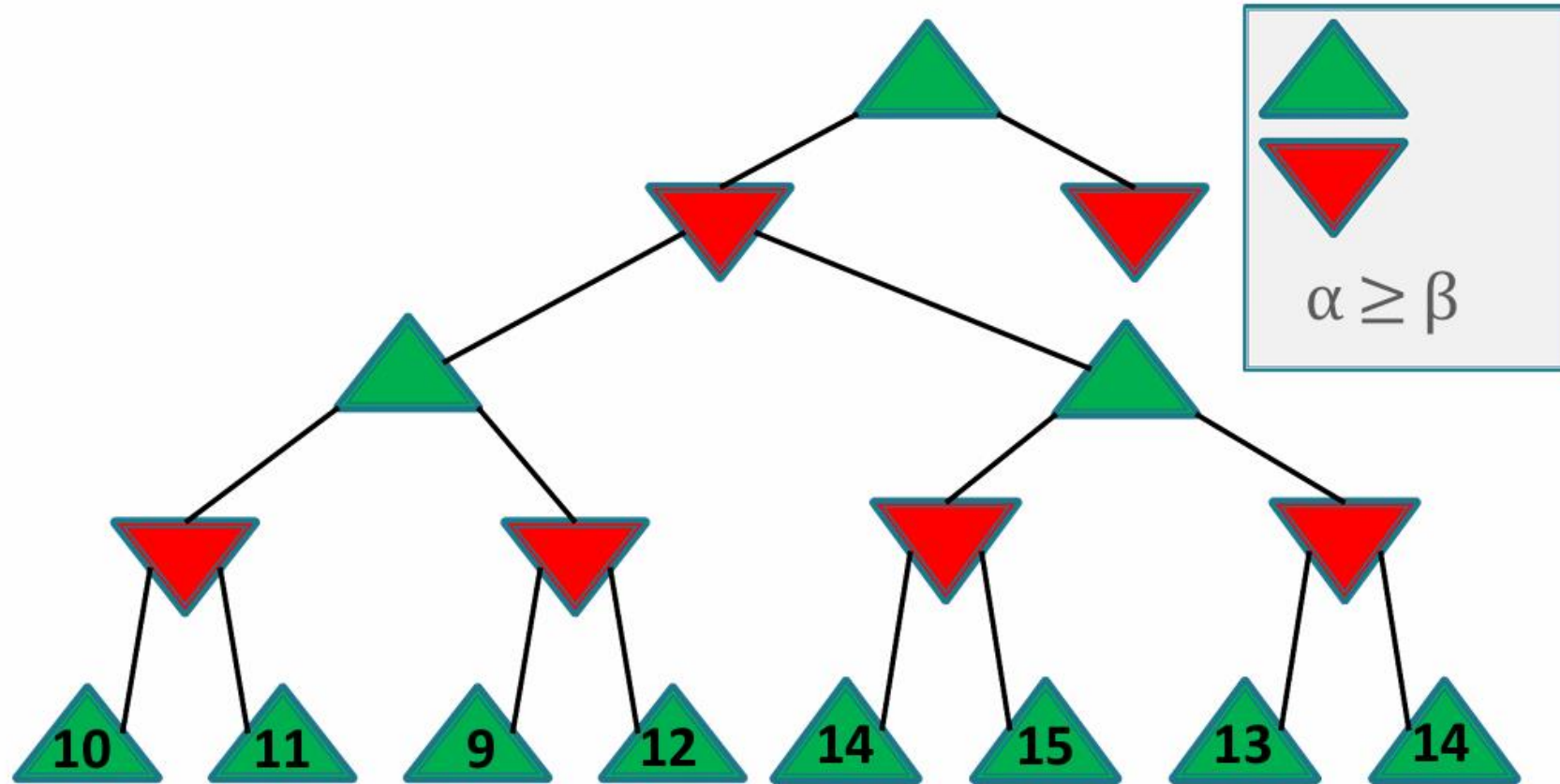
- Beta is the *upper bound on minimizing nodes.*

- Both alpha and beta get passed down the tree during the Minimax search.

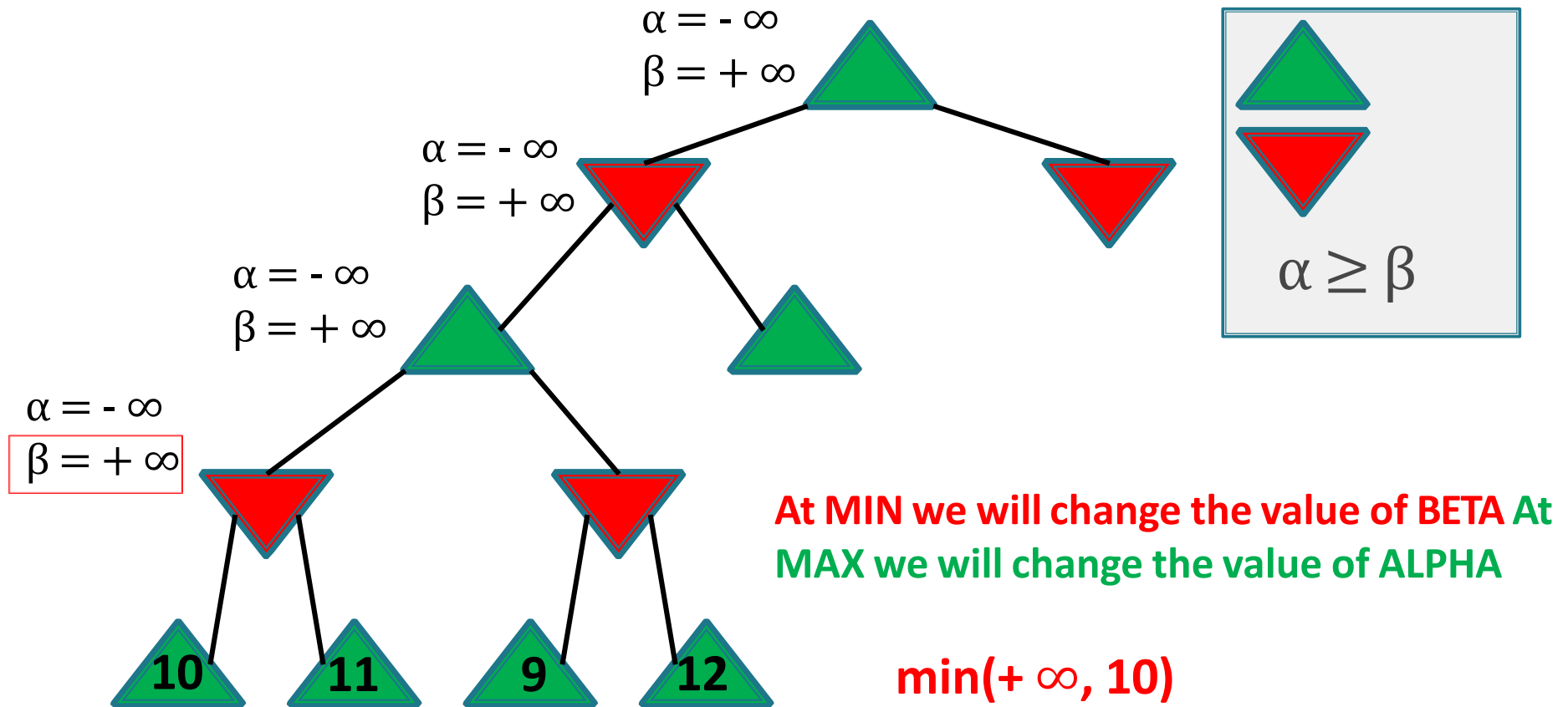
Mini-Max with Alpha Beta Pruning

- At minimizing nodes, we stop evaluating children if we get a child whose value is **less than the current lower bound (*alpha*)**.
- At maximizing nodes, we stop evaluating children as soon as we get a child whose value is **greater than the current upper bound (*beta*)**.
- Some branches will never be played by rational players since they include sub-optimal decisions (for either player)

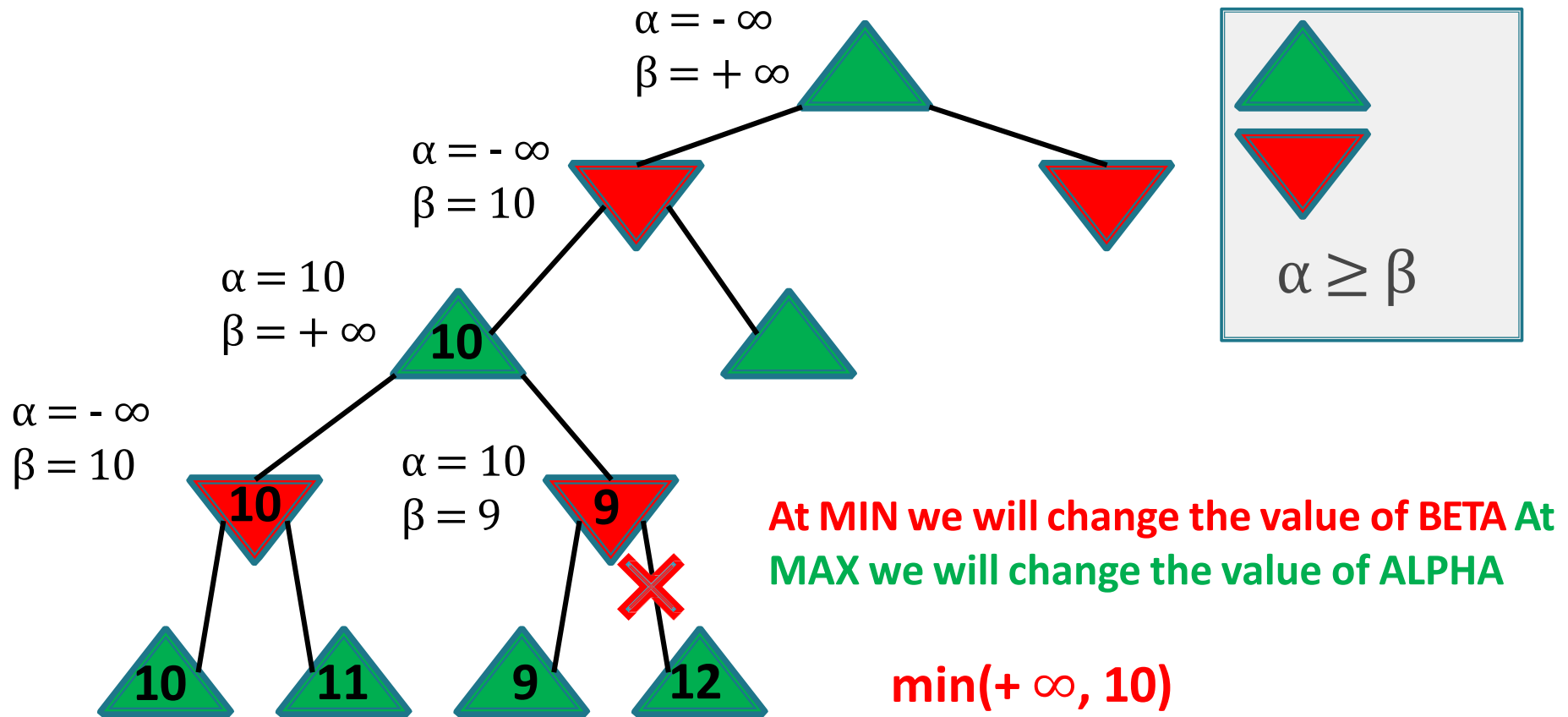
Alpha Beta Pruning Example



Alpha-Beta Pruning Example



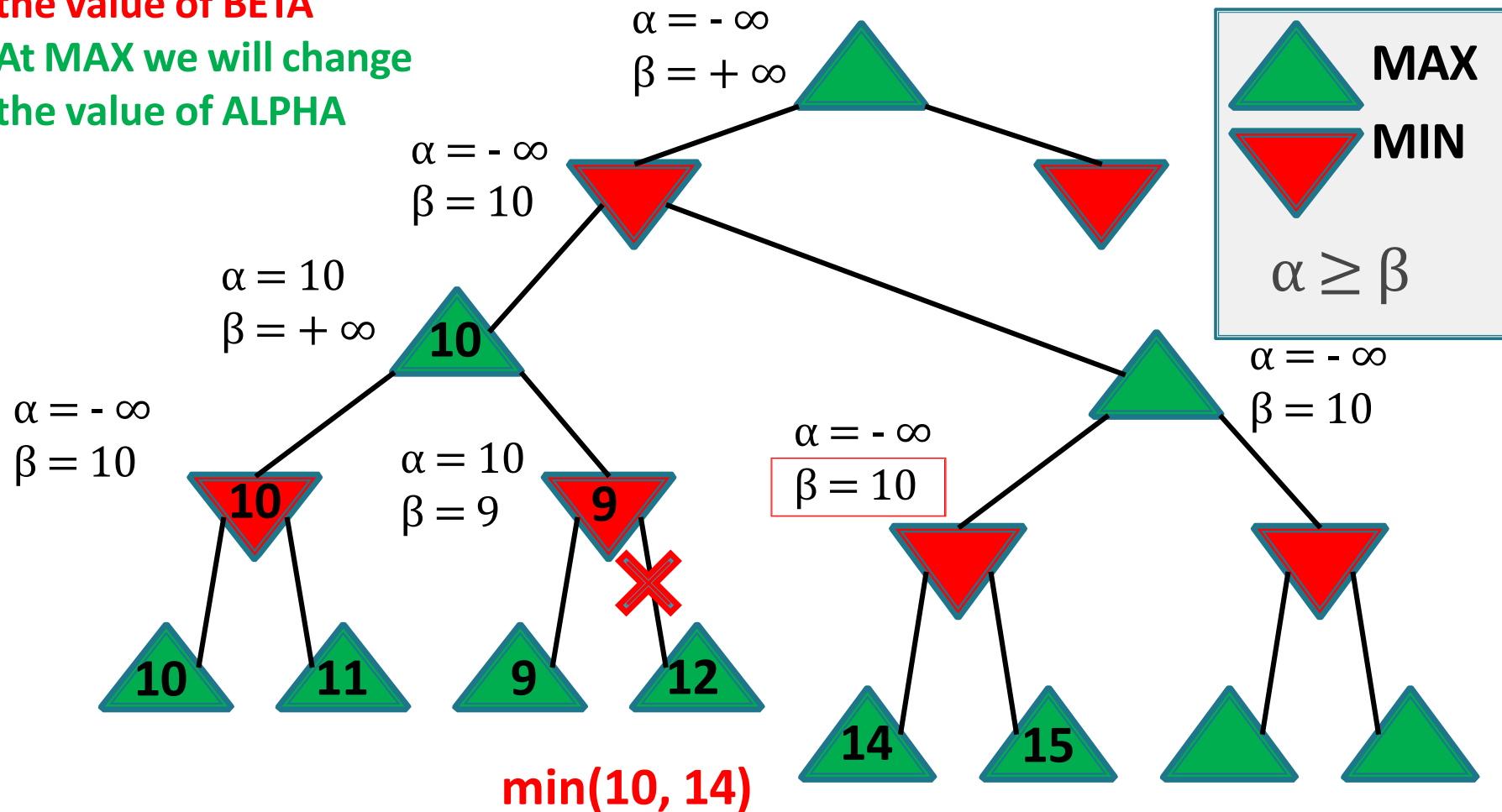
Alpha-Beta Pruning Example



Alpha-Beta Pruning Example

At MIN we will change
the value of BETA

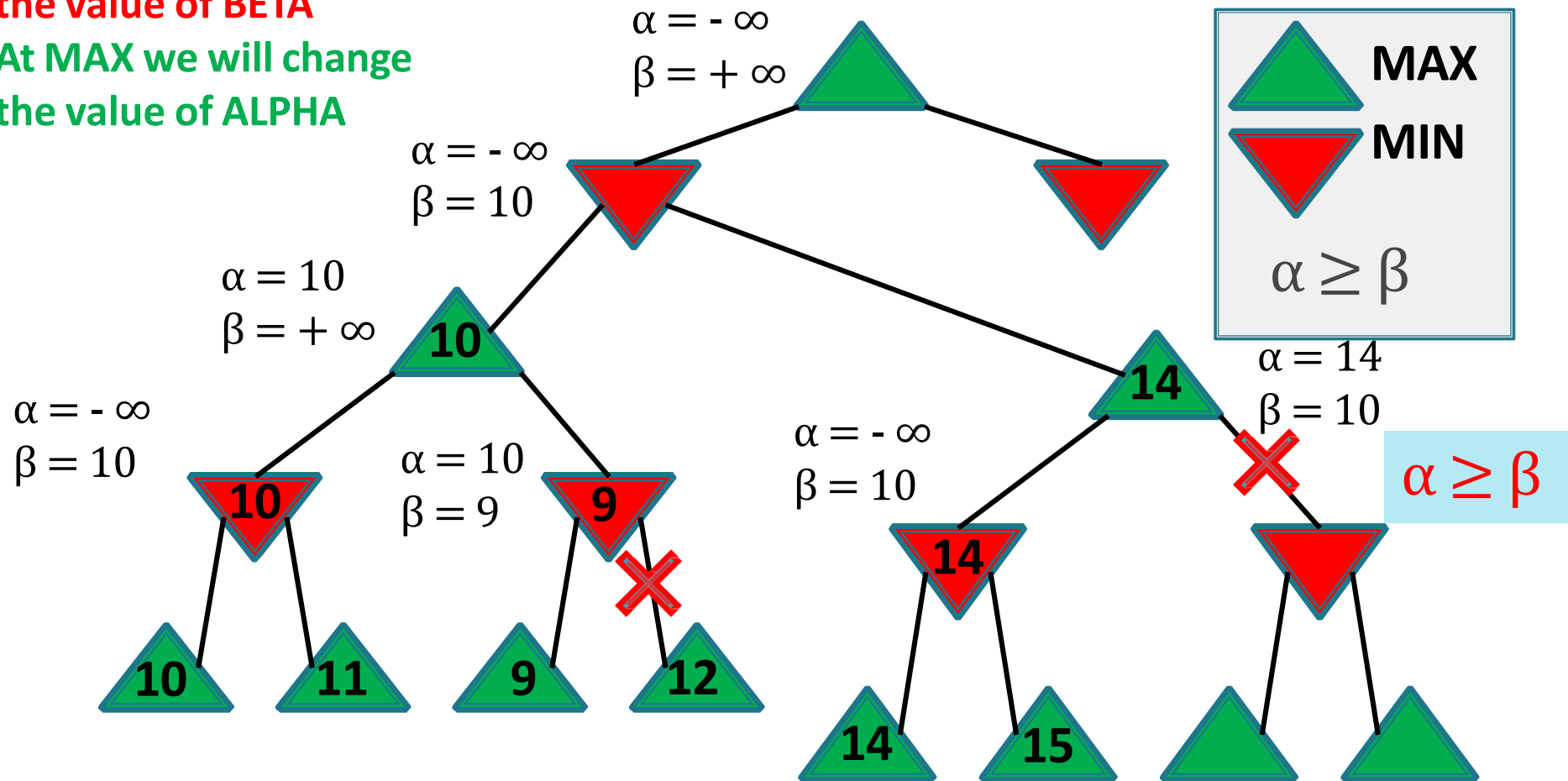
At MAX we will change
the value of ALPHA



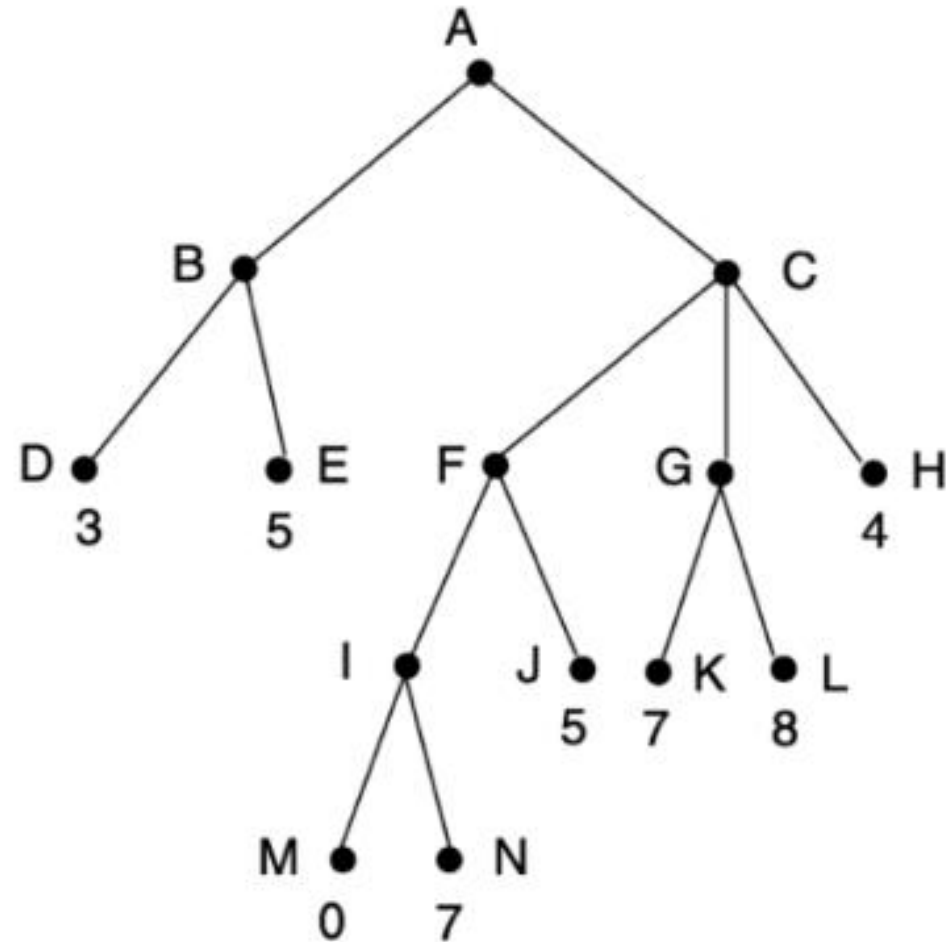
Alpha-Beta Pruning Example

At MIN we will change
the value of BETA

At MAX we will change
the value of ALPHA



Alpha Beta Pruning Example:



Alpha Beta Pruning: Effectiveness

- The effectiveness **depends on the order** in which children are visited.
- **In the best case**, the effective branching factor will be reduced from **b** to **$\sqrt{b}$** .
- **In an average case** (random values of leaves) the branching factor is reduced to $\frac{b}{\log b}$